In [43]:
```
import pandas as pd
```

In [44]:
```
df=pd.read_csv("/Users/tshmacm1171/Downloads/engineering_calculus_dataset
```

In [45]:
```
df.head(5)
```

Out[45]:

	Total_Hours_Studied	Tutorials_Attended	Classes_Missed	Assignments_Missed
0	132	4	5	0
1	116	3	2	1
2	136	10	5	1
3	158	3	6	0
4	114	11	9	2

data preprocessing

In [46]:
```
#dropping unnnecccesary column
df=df.drop(columns=["Random_Noise"])
```

In [47]:
```
df_new.head(4)
```

Out[47]:

	Total_Hours_Studied	Tutorials_Attended	Classes_Missed	Assignments_Missed
0	132	4	5	0
1	116	3	2	1
2	136	10	5	1
3	158	3	6	0

In [49]:
```
df["Performance_Level"].unique()
```

Out[49]:
```
array(['Average', 'Poor', 'Good'], dtype=object)
```

In [50]:
```
#checking for the duplicated rows

df.duplicated().any()
```

Out[50]:
```
np.False_
```

In [51]:
```
#checking the messing values

df.isnull().sum()
```

Out[51]:
```
Total_Hours_Studied     0
Tutorials_Attended      0
Classes_Missed          0
Assignments_Missed      0
Marks                   0
Study_Efficiency       10
Engagement_Score       10
Hours_x_Tutorials       0
Performance_Level       0
At_Risk                 0
dtype: int64
```

In [52]:
```
#for engagment we see the formula is marks/total_hours_studied
df["Study_Efficiency"]=df["Study_Efficiency"].fillna(df["Marks"]/df["Tota
```

In [53]:
```
# imputing a mean for the missing values in engagement
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="mean")

df["Engagement_Score"]= imputer.fit_transform(df[["Engagement_Score"]])
```

In [54]:
```
df.isnull().sum()
```

Out[54]:
```
Total_Hours_Studied    0
Tutorials_Attended     0
Classes_Missed         0
Assignments_Missed     0
Marks                  0
Study_Efficiency       0
Engagement_Score       0
Hours_x_Tutorials      0
Performance_Level      0
At_Risk                0
dtype: int64
```

In [55]:
```
df.head(5)
```

Out[55]:

	Total_Hours_Studied	Tutorials_Attended	Classes_Missed	Assignments_Missed
0	132	4	5	0
1	116	3	2	1
2	136	10	5	1
3	158	3	6	0
4	114	11	9	2

In [90]:
```
# Target
y = df['Marks']  # your target

# Features
X_f = df.drop('Marks', axis=1)  # drop target from features
```

In [91]:
```
# Numeric columns: everything except categorical
num_value = [col for col in X_f.columns if col not in ['Performance_Level
```

```
# Categorical columns
cat_value = ['Performance_Level']  # only features, NOT target
```

In [92]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

numeric_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(drop='first', sparse_output=False


preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, num_value),
        ('cat', categorical_transformer , cat_value)
    ],
    remainder='passthrough'  # keeps other columns
)
```

In [93]:
```
X_processed = preprocessor.fit_transform(X_f)
```

In [94]:
```
#convert the array back to pd

import pandas as pd

X_processed_df = pd.DataFrame(
    X_processed,
    columns=preprocessor.get_feature_names_out()
)
```

In [95]:
```
X_processed_df.head(10)
```

Out[95]:

	num__Total_Hours_Studied	num__Tutorials_Attended	num__Classes_Missed	nu
0	0.580873	-1.118017	0.2691	
1	-0.108335	-1.508250	-0.7659	
2	0.753174	1.223380	0.2691	
3	1.700834	-1.508250	0.6141	
4	-0.194486	1.613613	1.6491	
5	-0.194486	0.442914	1.6491	
6	1.743910	0.833147	-0.7659	
7	0.882401	0.833147	0.6141	
8	-0.452938	-0.727784	-0.7659	
9	0.623948	-1.118017	-1.1109	

In [96]:
```
# naming the variables

X=X_processed_df
```

In [100…
```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test= train_test_split(X, y, test_size=0.2, r
```

In [101…

```
from sklearn.linear_model import LinearRegression

model_c= LinearRegression()
model_c.fit(X_train, y_train)
```

Out[101…

```
▾ LinearRegression
▶ Parameters
```

In [102…

```
y_predicted=model_c.predict(X_test)
```

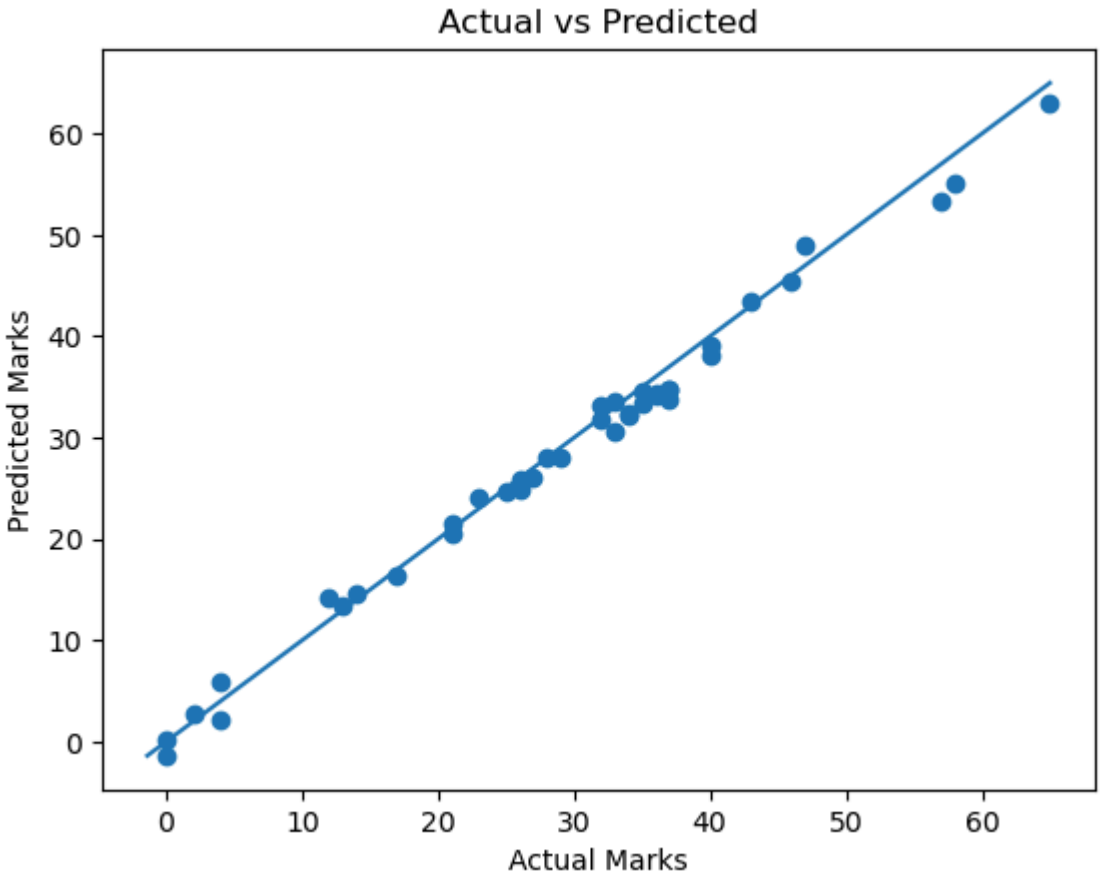
In [106…

```
# testing the metrics of our model

from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

mse=mean_squared_error(y_test, y_predicted)
r2=r2_score(y_test, y_predicted)
rmse=np.sqrt(mse)

print("Mean Square Error is :", mse)
print("R squared is :", r2)
print("Root Mean Square Error is: ", rmse)
```

```
Mean Square Error is : 2.469646033521472
R squared is : 0.9892861362644297
Root Mean Square Error is:  1.5715107487769442
```

In [107…

```
import matplotlib.pyplot as plt
import numpy as np

# Convert to arrays (prevents most errors)
y_test_array = np.array(y_test)
y_pred_array = np.array(y_predicted)

plt.scatter(y_test_array, y_pred_array)

# Perfect line
min_val = min(y_test_array.min(), y_pred_array.min())
max_val = max(y_test_array.max(), y_pred_array.max())
plt.plot([min_val, max_val], [min_val, max_val])

plt.xlabel("Actual Marks")
plt.ylabel("Predicted Marks")
plt.title("Actual vs Predicted")

plt.show()
```


Actual vs Predicted
Predicted Marks
Actual Marks
0
10
20
30
40
50
60

In [ ]: